

Software Engineering

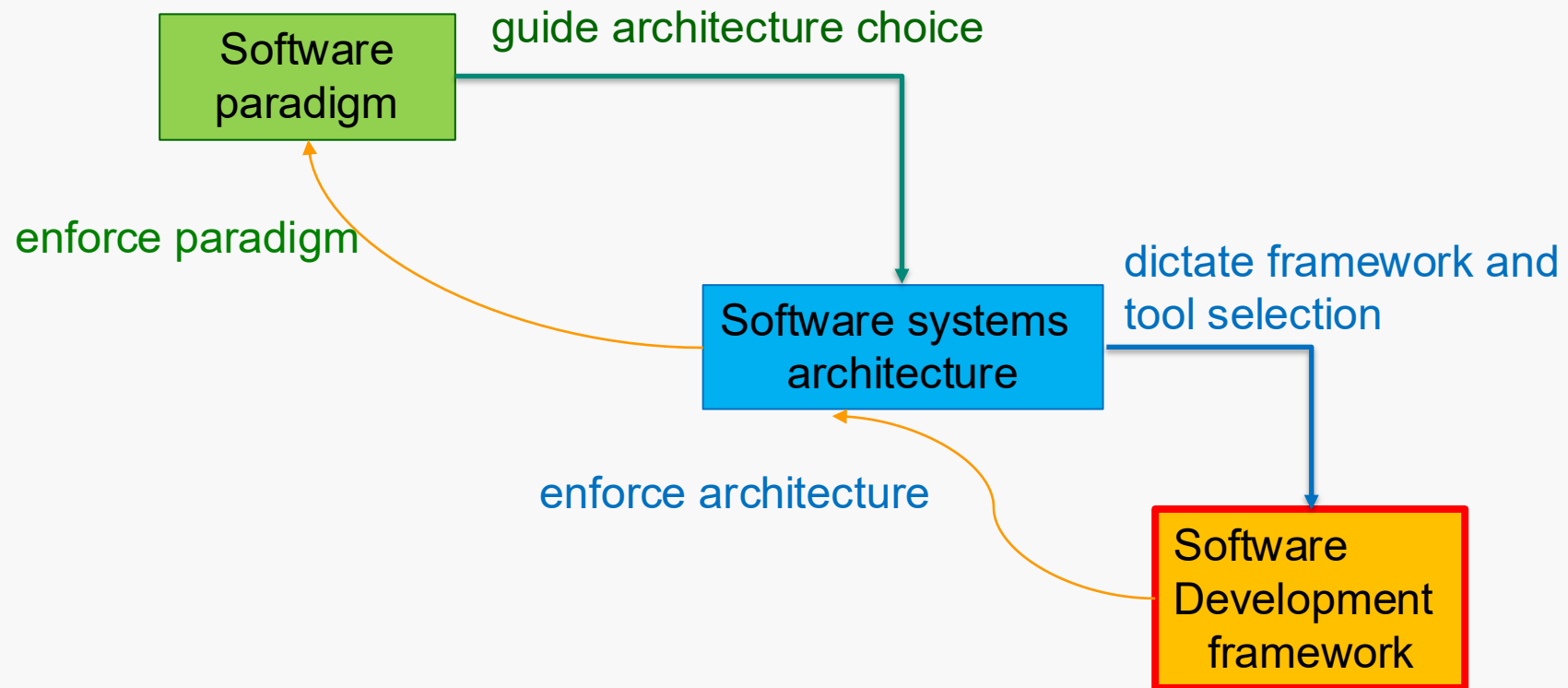
- Software Development Frameworks and Standards

Dr Lily Sun(Dr. Nisha Singh)

Learning Objectives

- ❑ Define key concepts related to software development frameworks and standards
- ❑ Describe commonly used software development models/tools and international standards in software engineering (e.g., ISO/IEC 12207, IEEE 830, CMMI)
- ❑ Explain the importance of using frameworks and standards for quality, efficiency, and compliance in coding
- ❑ Evaluate the benefits and challenges of adopting standardised development practices in real-world projects

SE Conceptual Flow



overall process to manage and execute software development project

Software development methodology

Software Development Framework

❑ Software Development Framework

- ✎ a structured platform of reusable components, tools, and guidelines
- ✎ aiding efficient design, development, testing, and deployment of software applications
- ✎ a structured approach for organising and managing software project

Purposes of Software Development Framework

❑ Accelerate development by reducing repetitive coding

- ✿ provide pre-built modules, e.g., **Django** high-level, open-source web framework written in Python
 - authentication system (login/logout, password management)
 - full functional admin dashboard from the data model
 - URL routing
 - database ORM (Object-Relational Mapping) – CRUD operation(s)
 - secure password hashing and user authentication
 - session and cookie management

Purposes of Software Development Framework (cont.)

- ❑ Accelerate development by reducing repetitive coding
 - 🌿 apply code generators, templates, and scaffolding commands speed up setup and development
 - 🌿 assist teams to focus on business logic rather than infrastructure code

Purposes of Software Development Framework (cont.)

❑ Enforce best practices and reduce errors

- 🦋 Prevent common programming mistakes, bugs, vulnerability
- 🦋 validating input as build-in capability
- 🦋 authentication and security default (safe password storage, safe database access,)
- 🦋 encouraging test-driven and modular design

Purposes of Software Development Framework (cont.)

❑ Facilitate teamwork through shared structure and conventions

🦊 predefine rules, file organisation, naming pattern

Example in Django

```
myapp/
├── models.py      # Data models
├── views.py       # Logic to handle requests
├── templates/     # HTML templates
├── forms.py       # Form definitions
└── urls.py        # URL routing
```

Purposes of Software Development Framework (cont.)

- ❑ Facilitate teamwork through shared structure and conventions

Framework Feature	Teamwork Benefit
Standard folder structure	Easy navigation and onboarding
Naming conventions	Consistent, readable code
Shared libraries & patterns	Avoid duplication, maintain compatibility
Modular design	Parallel development and reuse
Built-in documentation tools	Faster understanding and handoff

Purposes of Software Development Framework (cont.)

- ❑ Reinforce quality and maintainability through consistent patterns
 - 🌿 the use of proven design patterns leading to cleaner and more structured code
 - 🌿 predictable code structure makes it easier for new developers to understand and maintain the software
 - 🌿 aiding software development in an eco-business environment

Purposes of Software Development Framework (cont.)

- ❑ Reinforce quality and maintainability through consistent patterns (cont.)

Pattern/ Framework	Ideal For	Key Benefit
MVP	UI-focused apps (legacy Android)	Better testability than MVC
Layered (n-tier)	Enterprise apps	Clear separation of responsibilities
Clean Architecture	Long-term maintainability	Core logic is isolated and testable
Microservices	Scalable systems	Independent services, agile teams
Event-Driven	Real-time processing	Loosely coupled components
Serverless	Cloud-native apps	No infrastructure to manage

Examples of Web Development Framework

- frontend

Framework	Language	Common Use Cases	Industry Use
React.js	JavaScript	Web UIs, SPAs, dashboards	Very widely used by big tech and startups
Angular	TypeScript	Enterprise dashboards, SPAs	Popular in enterprise-level apps
Vue.js	JavaScript	Lightweight SPAs, UI interactions	Favored in startups and agencies

Examples of Web Development Framework

- backend

Framework	Language	Common Use Cases	Industry Use
Django	Python	Web apps, dashboards, data-driven apps	Widely used in startups, data science platforms
Spring Boot	Java	Enterprise apps, APIs, microservices	Popular in large enterprises, banks, telecom
Express.js	JavaScript	REST APIs, real-time apps (Node.js-based)	Tech startups, scalable backend services
Ruby on Rails	Ruby	MVPs, web apps with fast prototyping	Some startups and legacy projects
Laravel	PHP	Web apps, CMS, e-commerce	Popular in web agencies and small businesses
ASP.NET Core	C#	Enterprise web apps, Microsoft stack	Used in finance, government, enterprise IT

Examples of Mobile App Development Framework

Framework	Language	Use Case	Industry Use
React Native	JavaScript	Cross-platform mobile apps	Used by Facebook, Instagram, Walmart
Flutter	Dart	Cross-platform, native-like apps	Used by Google, Alibaba, startups
SwiftUI	Swift	iOS apps	Apple ecosystem apps
Jetpack Compose	Kotlin	Android apps	Google-endorsed Android development

Most Common Frameworks Used in Industry

Domain	Most Common Frameworks
Web fronted	React, Angular, Vue
Web backend	Django, Spring, Boot, Express.js
Mobile apps	ReactNative, Flutter
Desktop apps	Java FX, .NET, Electron
Enterprise software	Spring Boot, ASP.NET Core
Startups/MVPs	Django, Laravel, Node.js +Express

Key Software Development Models

- ❑ Waterfall – Linear and sequential
- ❑ Agile – Iterative and flexible
- ❑ Scrum – Agile methodology focused on sprints
- ❑ DevOps – Integrates development and operations
- ❑ Spiral Model – Combines prototyping and iterative refinement

Standards in Software Development

Ensuring quality, interoperability, and safety

Goal	What It Means	How Frameworks Support It	Example in Django
 Quality	Code correctness & maintainability	Testing tools, clean architecture, reusable code	<code>TestCase</code> , MVT pattern
 Interoperability	Works with other systems	APIs, data formats, external services	Django REST Framework
 Safety	Secure & protected against attacks	CSRF protection, input validation, auth systems	CSRF middleware, auth decorators

CSRF cross-site request forgery

Standards in Software Development (cont.)

❑ Reinforcing documentation, testing, and compliance

Aspect	What It Reinforces	Framework Features That Help	Django Example
■ Documentation	Understandability, maintainability	Auto-generated docs, docstrings, Swagger	<code>drf-yasg</code> , docstrings in models/views
✅ Testing	Code correctness and reliability	Test classes, test runners, CI tools	<code>TestCase</code> , <code>pytest-django</code>
⚖️ Compliance	Legal, security, and data standards	Secure defaults, auth systems, logging	Hashed passwords, CSRF protection

Software Engineering Standards

Standard	Organization	Description
<u>ISO/IEC 12207</u>	ISO/IEC	Software lifecycle processes
<u>IEEE 830</u>	IEEE	Software Requirements Specification
<u>IEEE 1016</u>	IEEE	Software Design Description
<u>ISO/IEC 250120</u>	ISO/IEC	Software product quality model
<u>CMMI</u>	CMMI Institute	Maturity model for process improvement

ISO/IEC 25010 – Software Quality Model

Key Quality Characteristics and What They Check:

Characteristic

What It Means / Example for Mobile Banking App

Functional Suitability

Software does what it's supposed to do.

Performance Efficiency

Software is fast and uses resources wisely.

Compatibility

Works well with other systems and devices

Usability

Easy for users to learn and use

Reliability

Software is stable and handles errors gracefully.

Security

Protects user data and prevents unauthorized access

Maintainability

Easy for developers to fix and update

Portability

Software can run in different environments

Challenges in Adopting Standards

❑ Resistance to change

- 👉 organisations or software development teams inherit the established habits, processes, or legacy mindsets
 - fear of the unknown – impact to individuals' roles, skills, or relevance in a changing environment
 - comfort with existing processes – long-standing workflows and experience may no longer fit
 - perceived threats to autonomy or competence – new systems or standards impose control or reveal deficiencies
- 👉 overcome resistance
 - effective change management strategies
 - clear communication
 - strong leadership are essential

Challenges in Adopting Standards (cont.)

❑ Upfront cost and time

- 🌿 implementing new standards in systems typically involves significant initial investment
 - 🌿 financial costs – New tools, infrastructure upgrades, or consulting fees can be expensive
 - 🌿 time investment – Training staff, redesigning workflows, and debugging integration issues can delay ongoing operations
 - 🌿 short-term productivity loss – During the transition, teams may experience confusion or delays as they adjust to the new way of working
- 🌿 these costs sometime hard to justify upfront without a clear return on investment (ROI) forecast

Challenges in Adopting Standards (cont.)

❑ Balancing agility with structure

- ✿ struggling to find the right balance between being agile and maintaining necessary structure
 - too much structure can slow down decision-making, reduce responsiveness, and stifle innovation
 - too much agility can lead to inconsistent practices, lack of accountability, and difficulty scaling operations
- ✿ overcome balancing
 - defining clear but flexible processes
 - encouraging iterative development while maintaining alignment with long-term goals
 - adopting software development frameworks that support innovation without sacrificing compliance or quality

Challenges in Adopting Standards (cont.)

❑ Integrating standards in legacy systems

- 🦋 legacy systems often pose a significant challenge when integrating new standards
 - incompatibility – older systems do not support modern protocols, languages, or integrations
 - lack of documentation – difficulties on understanding legacy systems' behaviour, plus developers have left
 - risk of disruption – changing something in a legacy environment can cause unanticipated outages or data issues
 - resource constraints – maintaining and upgrading legacy systems conflict with investment in new solutions
- 🦋 overcome this
 - adopt strategies such as gradual migration,
 - hybrid systems that bridge old and new technologies

CS2SE - Software Engineering and Professional Development - Semester 1



Quiz

